

Thiết kế Exception Handling cho Checked Exception chuẩn Production trong Spring Boot

1. Checked Exception là gì?

Trong Java, **Checked Exception** là loại exception bắt buộc developer phải xử lý bằng `try-catch` hoặc khai báo `throws`.

Ví dụ:

```
public void readFile(String path) throws IOException {  
    Files.readString(Path.of(path));  
}
```

Các checked exception phổ biến:

```
IOException  
SQLException  
ParseException  
ClassNotFoundException  
InterruptedException  
TimeoutException  
ExecutionException  
JAXBException  
JsonProcessingException  
FileNotFoundException  
MalformedURLException
```

Trong production, vấn đề không phải là “catch cho hết lỗi”, mà là thiết kế sao cho:

- Không leak lỗi kỹ thuật ra controller/client.
- Không mất stack trace gốc.
- Không làm sai transaction rollback.
- Không nuốt lỗi.
- Phân loại đúng lỗi business, system, external service, file, parsing, database.
- Trả response an toàn, thống nhất.
- Log đủ thông tin để debug.

2. Nguyên tắc production khi xử lý Checked Exception

2.1. Không để checked exception leak lung tung qua nhiều layer

Không nên để service public method throws nhiều exception kỹ thuật:

```
public OrderResponse importOrder(MultipartFile file)
    throws IOException, ParseException, SQLException {
    // logic
}
```

Vấn đề:

- Controller phải biết quá nhiều lỗi hạ tầng.
- API contract bị phụ thuộc implementation.
- Transaction rollback có thể không đúng.
- Code bị lan truyền `throws`.
- Khó mapping sang HTTP status/error code.

Nên convert checked exception sang custom application exception tại boundary phù hợp.

Ví dụ:

```
public OrderResponse importOrder(MultipartFile file) {
    try {
        return orderImportProcessor.process(file);
    } catch (IOException e) {
        throw new FileProcessingException("Cannot read order import file", e);
    } catch (ParseException e) {
        throw new InvalidFileFormatException("Invalid order import file
format", e);
    }
}
```

2.2. Luôn preserve root cause

Không nên:

```
catch (IOException e) {  
    throw new RuntimeException("File error");  
}
```

Vì mất stack trace gốc.

Nên:

```
catch (IOException e) {  
    throw new FileProcessingException("Cannot read file", e);  
}
```

Custom exception phải nhận `Throwable cause`.

2.3. Không catch Exception quá rộng nếu không cần

Không nên:

```
try {  
    processFile(file);  
} catch (Exception e) {  
    throw new RuntimeException(e);  
}
```

Nên catch theo nhóm cụ thể:

```
try {  
    processFile(file);  
} catch (IOException e) {  
    throw new FileProcessingException("File read failed", e);  
} catch (CsvValidationException e) {  
    throw new InvalidFormatException("Invalid CSV format", e);  
}
```

2.4. Không dùng `throws Exception` ở service/controller

Không nên:

```
@PostMapping("/import")
public ApiResponse<Void> importFile(MultipartFile file) throws Exception {
    orderService.importFile(file);
    return ApiResponse.success(null);
}
```

Nên:

```
@PostMapping("/import")
public ResponseEntity<ApiResponse<Void>> importFile(
    @RequestParam MultipartFile file
) {
    orderService.importFile(file);
    return ResponseEntity.ok(ApiResponse.success("Import completed", null));
}
```

Exception nên được xử lý qua custom exception và `GlobalExceptionHandler`.

2.5. Cẩn thận với transaction rollback

Spring `@Transactional` mặc định chỉ rollback với:

```
RuntimeException
Error
```

Không rollback mặc định với checked exception.

Ví dụ nguy hiểm:

```
@Transactional
public void importOrder(MultipartFile file) throws IOException {
    orderRepository.save(order);

    if (file.isEmpty()) {
        throw new IOException("File error");
    }
}
```

Trong case này, nếu chỉ throw `IOException`, transaction có thể không rollback như mong muốn.

Có 2 cách xử lý.

Cách 1: Convert sang RuntimeException custom.

```
@Transactional
public void importOrder(MultipartFile file) {
    try {
        orderRepository.save(order);
        readFile(file);
    } catch (IOException e) {
        throw new FileProcessingException("Cannot import order file", e);
    }
}
```

Cách 2: Khai báo rollbackFor.

```
@Transactional(rollbackFor = IOException.class)
public void importOrder(MultipartFile file) throws IOException {
    orderRepository.save(order);
    readFile(file);
}
```

Khuyến nghị production với Spring Boot:

Ưu tiên custom exception extends RuntimeException để transaction rollback tự nhiên.
Chỉ dùng rollbackFor khi thực sự muốn giữ checked exception ở method signature.

3. Kiến trúc xử lý Checked Exception chuẩn

3.1. Flow tổng quát

```
Controller
|
v
Service
|
v
Infrastructure / Adapter / Client / File / Parser
|
| throws IOException / SQLException / TimeoutException
v
```

```
Catch checked exception tại boundary
|
v
Convert sang custom RuntimeException có ErrorCode
|
v
GlobalExceptionHandler
|
v
Standard ErrorResponse
```

3.2. Package đề xuất

```
src/main/java/com/example/project
├── common
│   ├── exception
│   │   ├── ErrorCode.java
│   │   ├── BaseException.java
│   │   ├── FileProcessingException.java
│   │   ├── InvalidFileFormatException.java
│   │   ├── ExternalServiceException.java
│   │   ├── ExternalServiceTimeoutException.java
│   │   ├── DatabaseOperationException.java
│   │   ├── AsyncProcessingException.java
│   │   └── GlobalExceptionHandler.java
│   └── response
│       ├── ErrorResponse.java
│       └── ApiResponse.java
├── order
│   ├── service
│   ├── adapter
│   └── exception
└── payment
    ├── client
    └── exception
```

4. ErrorCode cho Checked Exception

```
package com.example.common.exception;

import org.springframework.http.HttpStatus;

public enum ErrorCode {

    // File
    FILE_READ_ERROR(
        "FILE_001",
        "Cannot read file",
        HttpStatus.BAD_REQUEST
    ),

    FILE_WRITE_ERROR(
        "FILE_002",
        "Cannot write file",
        HttpStatus.INTERNAL_SERVER_ERROR
    ),

    FILE_FORMAT_INVALID(
        "FILE_003",
        "Invalid file format",
        HttpStatus.UNPROCESSABLE_ENTITY
    ),

    FILE_TOO_LARGE(
        "FILE_004",
        "File size exceeds limit",
        HttpStatus.PAYLOAD_TOO_LARGE
    ),

    // Parsing
    JSON_PARSE_ERROR(
        "PARSE_001",
        "Invalid JSON format",
        HttpStatus.BAD_REQUEST
    ),

    DATE_PARSE_ERROR(
        "PARSE_002",
        "Invalid date format",
        HttpStatus.BAD_REQUEST
    ),
```

```

// External service
EXTERNAL_SERVICE_ERROR(
    "EXT_001",
    "External service error",
    HttpStatus.BAD_GATEWAY
),

EXTERNAL_SERVICE_TIMEOUT(
    "EXT_504",
    "External service timeout",
    HttpStatus.GATEWAY_TIMEOUT
),

// Database
DATABASE_ERROR(
    "DB_500",
    "Database error",
    HttpStatus.INTERNAL_SERVER_ERROR
),

// Async
ASYNC_PROCESSING_ERROR(
    "ASYNC_500",
    "Async processing error",
    HttpStatus.INTERNAL_SERVER_ERROR
),

// Common
INTERNAL_SERVER_ERROR(
    "COMMON_500",
    "Internal server error",
    HttpStatus.INTERNAL_SERVER_ERROR
);

private final String code;
private final String message;
private final HttpStatus status;

ErrorCode(String code, String message, HttpStatus status) {
    this.code = code;
    this.message = message;
    this.status = status;
}

public String getCode() {
    return code;
}

```



```
public String getMessage() {  
    return message;  
}  
  
public HttpStatus getStatus() {  
    return status;  
}  
}
```

5. BaseException

```
package com.example.common.exception;  
  
public abstract class BaseException extends RuntimeException {  
  
    private final ErrorCode errorCode;  
  
    protected BaseException(ErrorCode errorCode) {  
        super(errorCode.getMessage());  
        this.errorCode = errorCode;  
    }  
  
    protected BaseException(ErrorCode errorCode, String message) {  
        super(message);  
        this.errorCode = errorCode;  
    }  
  
    protected BaseException(ErrorCode errorCode, String message, Throwable  
cause) {  
        super(message, cause);  
        this.errorCode = errorCode;  
    }  
  
    protected BaseException(ErrorCode errorCode, Throwable cause) {  
        super(errorCode.getMessage(), cause);  
        this.errorCode = errorCode;  
    }  
  
    public ErrorCode getErrorCode() {  
        return errorCode;  
    }  
}
```

6. Custom exception cho Checked Exception

6.1. FileProcessingException

```
package com.example.common.exception;

public class FileProcessingException extends BaseException {

    public FileProcessingException(ErrorCode errorCode, String message,
    Throwable cause) {
        super(errorCode, message, cause);
    }

    public static FileProcessingException readFailed(Throwable cause) {
        return new FileProcessingException(
            ErrorCode.FILE_READ_ERROR,
            "Cannot read uploaded file",
            cause
        );
    }

    public static FileProcessingException writeFailed(Throwable cause) {
        return new FileProcessingException(
            ErrorCode.FILE_WRITE_ERROR,
            "Cannot write file",
            cause
        );
    }
}
```

6.2. InvalidFileFormatException

```
package com.example.common.exception;

public class InvalidFileFormatException extends BaseException {

    public InvalidFileFormatException(String message, Throwable cause) {
        super(ErrorCode.FILE_FORMAT_INVALID, message, cause);
    }
}
```

6.3. JsonParsingException

```
package com.example.common.exception;

public class JsonParsingException extends BaseException {

    public JsonParsingException(String message, Throwable cause) {
        super(ErrorCode.JSON_PARSE_ERROR, message, cause);
    }
}
```

6.4. ExternalServiceException

```
package com.example.common.exception;

public class ExternalServiceException extends BaseException {

    private final String serviceName;

    public ExternalServiceException(String serviceName, Throwable cause) {
        super(
            ErrorCode.EXTERNAL_SERVICE_ERROR,
            "External service error: " + serviceName,
            cause
        );
        this.serviceName = serviceName;
    }

    public ExternalServiceException(ErrorCode errorCode, String serviceName,
    Throwable cause) {
        super(
            errorCode,
            errorCode.getMessage() + ": " + serviceName,
            cause
        );
        this.serviceName = serviceName;
    }

    public String getServiceName() {
        return serviceName;
    }
}
```

6.5. ExternalServiceTimeoutException

```
package com.example.common.exception;

public class ExternalServiceTimeoutException extends BaseException {

    private final String serviceName;

    public ExternalServiceTimeoutException(String serviceName, Throwable cause)
    {
        super(
            ErrorCode.EXTERNAL_SERVICE_TIMEOUT,
            "External service timeout: " + serviceName,
            cause
        );
        this.serviceName = serviceName;
    }

    public String getServiceName() {
        return serviceName;
    }
}
```

7. GlobalExceptionHandler

```
package com.example.common.exception;

import com.example.common.response.ErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.slf4j.MDC;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.io.FileNotFoundException;
import java.io.IOException;
import java.sql.SQLException;
import java.text.ParseException;
import java.util.concurrent.TimeoutException;

@RestControllerAdvice
public class GlobalExceptionHandler {
```

```

@ExceptionHandler(BaseException.class)
public ResponseEntity<ErrorResponse> handleBaseException(
    BaseException ex,
    HttpServletRequest request
) {
    ErrorCode errorCode = ex.getErrorCode();
    HttpStatus status = errorCode.getStatus();

    ErrorResponse response = ErrorResponse.of(
        status.value(),
        status.name(),
        errorCode.getCode(),
        ex.getMessage(),
        request.getRequestURI(),
        getTraceId()
    );

    return ResponseEntity.status(status).body(response);
}

/**
 * Fallback cho IOException nếu có case bị leak ra controller.
 * Trong code production tốt, IOException nên được convert ở service/adapter
layer.
 */
@ExceptionHandler(FileNotFoundException.class)
public ResponseEntity<ErrorResponse> handleFileNotFound(
    FileNotFoundException ex,
    HttpServletRequest request
) {
    ErrorCode errorCode = ErrorCode.FILE_READ_ERROR;
    HttpStatus status = errorCode.getStatus();

    ErrorResponse response = ErrorResponse.of(
        status.value(),
        status.name(),
        errorCode.getCode(),
        "File not found",
        request.getRequestURI(),
        getTraceId()
    );

    return ResponseEntity.status(status).body(response);
}

@ExceptionHandler(IOException.class)
public ResponseEntity<ErrorResponse> handleIOException(

```

```

        IOException ex,
        HttpServletRequest request
    ) {
        ErrorCode errorCode = ErrorCode.FILE_READ_ERROR;
        HttpStatus status = errorCode.getStatus();

        ErrorResponse response = ErrorResponse.of(
            status.value(),
            status.name(),
            errorCode.getCode(),
            "I/O error while processing request",
            request.getRequestURI(),
            getTraceId()
        );

        return ResponseEntity.status(status).body(response);
    }

    @ExceptionHandler(ParseException.class)
    public ResponseEntity<ErrorResponse> handleParseException(
        ParseException ex,
        HttpServletRequest request
    ) {
        ErrorCode errorCode = ErrorCode.DATE_PARSE_ERROR;
        HttpStatus status = errorCode.getStatus();

        ErrorResponse response = ErrorResponse.of(
            status.value(),
            status.name(),
            errorCode.getCode(),
            "Invalid input format",
            request.getRequestURI(),
            getTraceId()
        );

        return ResponseEntity.status(status).body(response);
    }

    @ExceptionHandler(TimeoutException.class)
    public ResponseEntity<ErrorResponse> handleTimeoutException(
        TimeoutException ex,
        HttpServletRequest request
    ) {
        ErrorCode errorCode = ErrorCode.EXTERNAL_SERVICE_TIMEOUT;
        HttpStatus status = errorCode.getStatus();

        ErrorResponse response = ErrorResponse.of(
            status.value(),

```

```

        status.name(),
        errorCode.getCode(),
        "Request timeout while calling external service",
        request.getRequestURI(),
        getTraceId()
    );

    return ResponseEntity.status(status).body(response);
}

/**
 * SQLException thường đã được Spring wrap thành DataAccessException.
 * Handler này chỉ là fallback nếu project dùng JDBC thuần.
 */
@ExceptionHandler(SQLException.class)
public ResponseEntity<ErrorResponse> handleSQLException(
    SQLException ex,
    HttpServletRequest request
) {
    ErrorCode errorCode = ErrorCode.DATABASE_ERROR;
    HttpStatus status = errorCode.getStatus();

    ErrorResponse response = ErrorResponse.of(
        status.value(),
        status.name(),
        errorCode.getCode(),
        "Database error",
        request.getRequestURI(),
        getTraceId()
    );

    return ResponseEntity.status(status).body(response);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleUnexpectedException(
    Exception ex,
    HttpServletRequest request
) {
    ErrorCode errorCode = ErrorCode.INTERNAL_SERVER_ERROR;
    HttpStatus status = errorCode.getStatus();

    ErrorResponse response = ErrorResponse.of(
        status.value(),
        status.name(),
        errorCode.getCode(),
        errorCode.getMessage(),
        request.getRequestURI(),

```

```

        getTraceId()
    );

    return ResponseEntity.status(status).body(response);
}

private String getTraceId() {
    return MDC.get("traceId");
}
}

```

Lưu ý quan trọng:

GlobalExceptionHandler catch IOException/SQLException chỉ nên là fallback. Best practice là convert checked exception sang custom runtime exception tại service/adaptor layer.

8. ErrorResponse DTO

```

package com.example.common.response;

import com.fasterxml.jackson.annotation.JsonInclude;
import java.time.Instant;
import java.util.List;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class ErrorResponse {

    private boolean success;
    private Instant timestamp;
    private int status;
    private String error;
    private String code;
    private String message;
    private String path;
    private String traceId;
    private List<FieldErrorResponse> details;

    public static ErrorResponse of(
        int status,
        String error,
        String code,

```



```

        String message,
        String path,
        String traceId
    ) {
        ErrorResponse response = new ErrorResponse();
        response.success = false;
        response.timestamp = Instant.now();
        response.status = status;
        response.error = error;
        response.code = code;
        response.message = message;
        response.path = path;
        response.traceId = traceId;
        return response;
    }

    public boolean isSuccess() {
        return success;
    }

    public Instant getTimestamp() {
        return timestamp;
    }

    public int getStatus() {
        return status;
    }

    public String getError() {
        return error;
    }

    public String getCode() {
        return code;
    }

    public String getMessage() {
        return message;
    }

    public String getPath() {
        return path;
    }

    public String getTraceId() {
        return traceId;
    }

```

```
public List<FieldErrorResponse> getDetails() {  
    return details;  
}  
}
```

9. Case 1: IOException khi upload/import file

9.1. Không nên làm

```
@Transactional  
public void importUsers(MultipartFile file) throws IOException {  
    List<UserImportRow> rows = csvParser.parse(file.getInputStream());  
    userRepository.saveAll(mapToUsers(rows));  
}
```

Vấn đề:

- Service leak `IOException`.
- Transaction có thể không rollback nếu `IOException` được throw ra ngoài.
- Controller bị ép xử lý lỗi file.
- Không có error code rõ ràng.

9.2. Cách làm production

```
@Service  
@RequiredArgsConstructor  
public class UserImportService {  
  
    private final UserRepository userRepository;  
    private final CsvUserParser csvUserParser;  
  
    @Transactional  
    public void importUsers(MultipartFile file) {  
        validateFile(file);  
  
        List<UserImportRow> rows = parseFile(file);  
  
        List<User> users = rows.stream()  
            .map(this::toEntity)  
            .toList();  
    }  
}
```

```

        userRepository.saveAll(users);
    }

    private void validateFile(MultipartFile file) {
        if (file == null || file.isEmpty()) {
            throw new InvalidFormatException(
                "Uploaded file is empty",
                null
            );
        }

        String originalFilename = file.getOriginalFilename();

        if (originalFilename == null || !originalFilename.endsWith(".csv")) {
            throw new InvalidFormatException(
                "Only CSV file is supported",
                null
            );
        }
    }

    private List<UserImportRow> parseFile(MultipartFile file) {
        try {
            return csvUserParser.parse(file.getInputStream());
        } catch (IOException e) {
            throw FileProcessingException.readFailed(e);
        } catch (CsvValidationException e) {
            throw new InvalidFormatException(
                "Invalid CSV file format",
                e
            );
        }
    }

    private User toEntity(UserImportRow row) {
        User user = new User();
        user.setEmail(row.getEmail());
        user.setFullName(row.getFullName());
        return user;
    }
}

```

10. Case 2: JsonProcessingException

Jackson thường throw checked exception như `JsonProcessingException`.

10.1. Không nên làm

```
public PaymentCallback parse(String rawBody) throws JsonProcessingException {  
    return objectMapper.readValue(rawBody, PaymentCallback.class);  
}
```

10.2. Nên làm

```
@Component  
@RequiredArgsConstructor  
public class PaymentCallbackParser {  
  
    private final ObjectMapper objectMapper;  
  
    public PaymentCallback parse(String rawBody) {  
        try {  
            return objectMapper.readValue(rawBody, PaymentCallback.class);  
        } catch (JsonProcessingException e) {  
            throw new JsonParsingException(  
                "Invalid payment callback payload",  
                e  
            );  
        }  
    }  
}
```

Response trả về client:

```
{  
  "success": false,  
  "timestamp": "2026-06-04T10:15:30Z",  
  "status": 400,  
  "error": "BAD_REQUEST",  
  "code": "PARSE_001",  
  "message": "Invalid payment callback payload",  
  "path": "/api/payment/callback",  
  "traceId": "b4fd190b71294488"  
}
```

11. Case 3: SQLException

Nếu dùng Spring Data JPA hoặc JdbcTemplate, thường không cần catch `SQLException` trực tiếp vì Spring convert sang `DataAccessException`, là unchecked exception.

11.1. JDBC thuần không nên làm

```
public User findUser(Long id) throws SQLException {
    Connection connection = dataSource.getConnection();
    PreparedStatement statement = connection.prepareStatement("select * from
users where id = ?");
    statement.setLong(1, id);
    ResultSet rs = statement.executeQuery();
    // mapping
}
```

11.2. Nên dùng JdbcTemplate

```
@Repository
@RequiredArgsConstructor
public class UserJdbcRepository {

    private final JdbcTemplate jdbcTemplate;

    public User findById(Long id) {
        try {
            return jdbcTemplate.queryForObject(
                "select id, email, full_name from users where id = ?",
                this::mapRow,
                id
            );
        } catch (EmptyResultDataAccessException e) {
            throw new ResourceNotFoundException(
                ErrorCode.USER_NOT_FOUND,
                "User not found with id: " + id
            );
        } catch (DataAccessException e) {
            throw new DatabaseOperationException(
                "Cannot query user by id",
                e
            );
        }
    }
}
```

```

private User mapRow(ResultSet rs, int rowNum) throws SQLException {
    User user = new User();
    user.setId(rs.getLong("id"));
    user.setEmail(rs.getString("email"));
    user.setFullName(rs.getString("full_name"));
    return user;
}
}

```

`SQLException` trong `mapRow` được Spring wrap thành `DataAccessException`.

12. Case 4: InterruptedException

`InterruptedException` là checked exception đặc biệt. Không được nuốt interrupt signal.

12.1. Không nên làm

```

try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    log.warn("Thread interrupted");
}

```

Sai vì interrupt flag bị clear và không restore lại.

12.2. Nên làm

```

try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    throw new AsyncProcessingException(
        "Thread was interrupted while processing task",
        e
    );
}

```

Custom exception:

```

package com.example.common.exception;

```

```
public class AsyncProcessingException extends BaseException {

    public AsyncProcessingException(String message, Throwable cause) {
        super(ErrorCode.ASYNC_PROCESSING_ERROR, message, cause);
    }
}
```

Nguyên tắc:

Khi catch InterruptedException:

1. Gọi Thread.currentThread().interrupt()
2. Sau đó throw custom exception hoặc return gracefully

13. Case 5: TimeoutException / ExecutionException trong CompletableFuture

13.1. Ví dụ production

```
public PaymentResult chargeWithTimeout(PaymentRequest request) {
    CompletableFuture<PaymentResult> future =
        CompletableFuture.supplyAsync(() -> paymentClient.charge(request));

    try {
        return future.get(3, TimeUnit.SECONDS);
    } catch (TimeoutException e) {
        future.cancel(true);
        throw new ExternalServiceTimeoutException("PAYMENT_SERVICE", e);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        throw new AsyncProcessingException(
            "Payment processing was interrupted",
            e
        );
    } catch (ExecutionException e) {
        Throwable cause = e.getCause();

        if (cause instanceof BaseException baseException) {
            throw baseException;
        }

        throw new ExternalServiceException("PAYMENT_SERVICE", cause);
    }
}
```

```
}  
}
```

Điểm quan trọng:

- `TimeoutException` map sang 504.
- `InterruptedException` phải restore interrupt flag.
- `ExecutionException` cần unwrap `getCause()`.
- Không trả message gốc ra client.

14. Case 6: Checked exception từ external SDK

Nhiều SDK payment, email, storage có checked exception.

Ví dụ giả định:

```
public void sendEmail(EmailRequest request) {  
    try {  
        emailSdk.send(request);  
    } catch (EmailSdkTimeoutException e) {  
        throw new ExternalServiceTimeoutException("EMAIL_SERVICE", e);  
    } catch (EmailSdkException e) {  
        throw new ExternalServiceException("EMAIL_SERVICE", e);  
    }  
}
```

Không nên để service/application layer biết exception riêng của SDK:

```
public void sendInvoiceEmail(...) throws EmailSdkException
```

Vì khi đổi provider từ SendGrid sang SES, toàn bộ service bị ảnh hưởng.

Nên cô lập SDK exception trong adapter/client layer.

15. Case 7: Checked exception khi gọi REST bằng Java HTTP Client

```
@Component
@RequiredArgsConstructor
public class ShippingClient {

    private final HttpClient httpClient;
    private final ObjectMapper objectMapper;

    public ShippingFeeResponse getShippingFee(ShippingFeeRequest request) {
        try {
            HttpRequest httpRequest = buildRequest(request);

            HttpResponse<String> response = httpClient.send(
                httpRequest,
                HttpResponse.BodyHandlers.ofString()
            );

            if (response.statusCode() >= 500) {
                throw new ExternalServiceException(
                    "SHIPPING_SERVICE",
                    new RuntimeException("Shipping service returned 5xx")
                );
            }

            return objectMapper.readValue(
                response.body(),
                ShippingFeeResponse.class
            );

        } catch (HttpTimeoutException e) {
            throw new ExternalServiceTimeoutException("SHIPPING_SERVICE", e);
        } catch (IOException e) {
            throw new ExternalServiceException("SHIPPING_SERVICE", e);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new ExternalServiceException("SHIPPING_SERVICE", e);
        }
    }

    private HttpRequest buildRequest(ShippingFeeRequest request) {
        // build HTTP request
        return null;
    }
}
```

```
}  
}
```

16. Mapping Checked Exception sang HTTP Status

Checked Exception	Nguyên nhân	Custom Exception	HTTP Status
IOException khi đọc upload file	file lỗi, stream lỗi	FileProcessingException	400 / 500
FileNotFoundException	file không tồn tại	FileProcessingException	404 / 400
JsonProcessingException	JSON sai format	JsonParsingException	400
ParseException	date/input format sai	InvalidFormatException	400
SQLException	database lỗi	DatabaseOperationException	500
TimeoutException	external timeout	ExternalServiceTimeoutException	504
InterruptedException	thread bị interrupt	AsyncProcessingException	500
ExecutionException	async task fail	unwrap cause	tùy cause
JAXBException	XML parse/generate lỗi	InvalidFileFormatException / ExternalServiceException	400 / 502
GeneralSecurityException	crypto/signature lỗi	SecurityProcessingException	400 / 500

17. Logging strategy cho Checked Exception

17.1. Log ở đâu?

Khuyến nghị:

- GlobalExceptionHandler log lỗi tổng quát.
- Adapter/client layer log context kỹ thuật nếu cần.
- Không log trùng stack trace ở nhiều layer.

Nếu đã log stack trace ở `GlobalExceptionHandler`, ở service chỉ nên bổ sung context nếu thực sự cần.

17.2. Log level

Loại checked exception	Log level
File format invalid do user upload	INFO / WARN
JSON invalid do client gửi sai	INFO / WARN
External service timeout	WARN
External service IOException	WARN / ERROR
Database SQLException	ERROR
Unexpected checked exception	ERROR
InterruptedException	WARN
ParseException do input sai	INFO / WARN

17.3. Ví dụ log trong handler

```
@ExceptionHandler(BaseException.class)
public ResponseEntity<ErrorResponse> handleBaseException(
    BaseException ex,
    HttpServletRequest request
) {
    ErrorCode errorCode = ex.getErrorCode();
    HttpStatus status = errorCode.getStatus();

    if (status.is5xxServerError()) {
        log.error(
            "Application error. code={}, path={}, traceId={}",
            errorCode.getCode(),
            request.getRequestURI(),
            MDC.get("traceId"),
            ex
        );
    } else {
```

```

        log.warn(
            "Client/business error. code={}, path={}, traceId={},
message={}",
            errorCode.getCode(),
            request.getRequestURI(),
            MDC.get("traceId"),
            ex.getMessage()
        );
    }

    ErrorResponse response = ErrorResponse.of(
        status.value(),
        status.name(),
        errorCode.getCode(),
        ex.getMessage(),
        request.getRequestURI(),
        MDC.get("traceId")
    );

    return ResponseEntity.status(status).body(response);
}

```

18. Khi nào giữ Checked Exception?

Không phải lúc nào cũng convert sang runtime. Có một số case có thể giữ checked exception:

18.1. Library/API nội bộ muốn ép caller xử lý

Ví dụ module export file:

```

public interface FileExporter {

    ExportedFile export(ExportRequest request) throws ExportFailedException;
}

```

Nếu `ExportFailedException` là checked exception, caller bắt buộc phải xử lý.

Nhưng khi đi vào Spring service/controller, nên convert sang application exception.

18.2. Batch job có flow recover rõ ràng

Ví dụ:

```
try {
    importRow(row);
} catch (InvalidRowException e) {
    failedRows.add(row.toFailedResult(e.getMessage()));
}
```

Ở đây checked exception có thể hợp lý vì lỗi từng dòng không làm fail toàn bộ job.

18.3. Domain thật sự có recoverable error

Ví dụ:

```
try {
    reserveSeat(command);
} catch (SeatTemporarilyLockedException e) {
    retryLater(command);
}
```

Nếu caller có hành động recover rõ ràng, checked exception có thể dùng được.

19. Khi nào nên convert sang RuntimeException?

Nên convert khi:

- Lỗi là technical infrastructure error.
- Lỗi không recover được ở caller hiện tại.
- Lỗi cần rollback transaction.
- Lỗi cần map sang HTTP response.
- Lỗi đến từ external SDK/database/file system.
- Không muốn leak dependency exception ra service/controller.

Ví dụ nên convert:

```
IOException
SQLException
```

```
JsonProcessingException
TimeoutException
JAXBException
GeneralSecurityException
```

20. Anti-pattern cần tránh

20.1. Nuốt exception

```
try {
    processFile(file);
} catch (IOException e) {
    log.error("File error");
}
```

Sai vì flow vẫn tiếp tục dù xử lý file thất bại.

20.2. Throw RuntimeException chung chung

```
catch (IOException e) {
    throw new RuntimeException(e);
}
```

Sai vì không có error code, không map được HTTP status rõ ràng.

20.3. Mất root cause

```
catch (IOException e) {
    throw new FileProcessingException("Cannot process file", null);
}
```

Sai vì mất stack trace.

20.4. Dùng Lombok @SneakyThrows

```
@SneakyThrows
public void importFile(MultipartFile file) {
    process(file);
}
```

Không nên dùng trong production service layer vì làm exception flow không rõ ràng, khó review và khó kiểm soát transaction.

20.5. Khai báo throws Exception

```
public void process() throws Exception {
}
```

Không nên vì quá rộng, caller không biết phải xử lý gì.

20.6. Catch rồi rethrow sai loại

```
catch (JsonProcessingException e) {
    throw new DatabaseOperationException("Database error", e);
}
```

Sai vì làm sai bản chất lỗi, gây debug khó.

21. Checklist production cho Checked Exception

Design

- Không leak checked exception kỹ thuật ra controller.
- Có custom runtime exception tương ứng.
- Có ErrorCode rõ ràng.
- Có HTTP status phù hợp.
- Có preserve root cause.
- Không dùng throws Exception.

Transaction

- Checked exception không rollback mặc định.
- Dùng custom runtime exception hoặc `rollbackFor`.
- Không để transaction commit khi xử lý file/payment/import thất bại.

Logging

- 5xx log ERROR kèm stack trace.
- 4xx do user input log INFO/WARN.
- Có traceId.
- Không log dữ liệu nhạy cảm.
- Không log trùng stack trace quá nhiều layer.

External service

- Timeout map 504.
- Service unavailable map 503.
- Bad gateway map 502.
- SDK checked exception được wrap ở adapter layer.
- Không leak SDK exception ra business service.

Async

- `InterruptedException` phải restore interrupt flag.
- `ExecutionException` phải unwrap cause.
- Timeout phải cancel task nếu phù hợp.

File/Parsing

- File sai format map 400/422.
- File quá lớn map 413.
- File system/internal storage error map 500.
- JSON/XML parse error map 400.

22. Câu trả lời phỏng vấn mẫu

Trong Java, checked exception là exception bắt buộc phải catch hoặc khai báo throws, ví dụ như `IOException`, `SQLException`, `ParseException`, `TimeoutException`, `InterruptedException`. Tuy nhiên trong Spring Boot production, em không để các checked exception kỹ thuật này leak trực tiếp từ infrastructure layer lên controller.

Cách em thường thiết kế là catch checked exception tại boundary phù hợp, ví dụ file adapter, external client, parser hoặc repository dùng JDBC thuần. Sau đó em convert nó sang custom application exception extends

`RuntimeException`, ví dụ `FileProcessingException`, `InvalidFileFormatException`, `ExternalServiceTimeoutException`, `DatabaseOperationException`. Các exception này có `ErrorCode`, HTTP status tương ứng và preserve root cause bằng cách truyền `Throwable cause`.

Lý do em convert sang runtime exception là vì Spring `@Transactional` mặc định chỉ rollback với `RuntimeException` và `Error`, không rollback với checked exception. Nếu để service method throw `IOException` mà không khai báo `rollbackFor`, có thể xảy ra tình huống dữ liệu đã save vào DB nhưng lỗi file xảy ra sau đó mà transaction không rollback. Vì vậy trong service transaction, em ưu tiên throw custom runtime exception hoặc nếu bắt buộc giữ checked exception thì khai báo `@Transactional(rollbackFor = IOException.class)`.

Với `GlobalExceptionHandler`, em vẫn có thể khai báo handler cho `IOException`, `SQLException`, `TimeoutException` như một lớp fallback, nhưng best practice là không để các lỗi này đi tới handler trực tiếp. Handler chính nên xử lý `BaseException` và map sang `ErrorResponse` thống nhất gồm `status`, `code`, `message`, `path`, `traceId`, `timestamp`.

Một case đặc biệt là `InterruptedException`. Khi catch lỗi này, em luôn gọi `Thread.currentThread().interrupt()` để restore interrupt flag, sau đó mới throw custom exception hoặc kết thúc xử lý gracefully. Với `ExecutionException`, em unwrap `getCause()` để lấy lỗi thật bên trong. Với `TimeoutException`, em map sang 504 Gateway Timeout và có thể cancel task nếu phù hợp.

Tóm lại, với checked exception trong production, nguyên tắc của em là không leak lỗi kỹ thuật, không mất root cause, không làm sai transaction rollback, map lỗi sang error code rõ ràng, log đúng level và trả response an toàn cho client.

23. Template ngắn gọn nên áp dụng

Service

```
@Transactional
public void importOrders(MultipartFile file) {
    try {
        List<OrderImportRow> rows = parser.parse(file.getInputStream());
        saveOrders(rows);
    } catch (IOException e) {
        throw FileProcessingException.readFailed(e);
    } catch (InvalidFormatException e) {
        throw new InvalidFileFormatException("Invalid order import file", e);
    }
}
```

Custom exception

```
public class FileProcessingException extends BaseException {

    public FileProcessingException(ErrorCode errorCode, String message,
    Throwable cause) {
        super(errorCode, message, cause);
    }

    public static FileProcessingException readFailed(Throwable cause) {
        return new FileProcessingException(
            ErrorCode.FILE_READ_ERROR,
            "Cannot read uploaded file",
            cause
        );
    }
}
```

Handler

```
@ExceptionHandler(BaseException.class)
public ResponseEntity<ErrorResponse> handleBaseException(
    BaseException ex,
    HttpServletRequest request
) {
    ErrorCode errorCode = ex.getErrorCode();
    HttpStatus status = errorCode.getStatus();

    ErrorResponse response = ErrorResponse.of(
        status.value(),
        status.name(),
        errorCode.getCode(),
        ex.getMessage(),
        request.getRequestURI(),
        MDC.get("traceId")
    );

    return ResponseEntity.status(status).body(response);
}
```

Transaction fallback nếu vẫn giữ checked exception

```
@Transactional(rollbackFor = IOException.class)
public void importOrders(MultipartFile file) throws IOException {
    // processing
}
```

24. Kết luận

Thiết kế checked exception chuẩn production trong Spring Boot nên đi theo hướng:

```
Checked Exception kỹ thuật
    ↓
Catch tại adapter/service boundary
    ↓
Wrap thành custom RuntimeException có ErrorCode
    ↓
GlobalExceptionHandler map sang ErrorResponse
    ↓
Client nhận response an toàn, thống nhất
```

Không nên để `IOException`, `SQLException`, `ParseException`, `TimeoutException` lan truyền tự do qua nhiều layer. Checked exception cần được xử lý có chủ đích, preserve root cause, đảm bảo transaction rollback đúng và hỗ trợ logging/monitoring trong production.